House Price Prediction Using Machine Learning Algorithms

Introduction:

In this project we have implemented twelve kinds of machine learning algorithms. This project helps in understanding how different factors like size, location, and condition affect house prices. Machine Learning models analyze patterns in historical data to make predictions. It is widely used in real estate industries. This project demonstrates how data-driven approaches can improve decision-making.

About Dataset:

The dataset used is kc_house_data.csv, which contains housing-related information. It includes multiple features describing the properties of houses. Some features may require cleaning or transformation. The dataset size is large enough for training models. We have implemented the below algorithms.

Packages We Have Used:

In this project we have used the following packages and installed necessary requirements for all twelve machine learning algorithms.

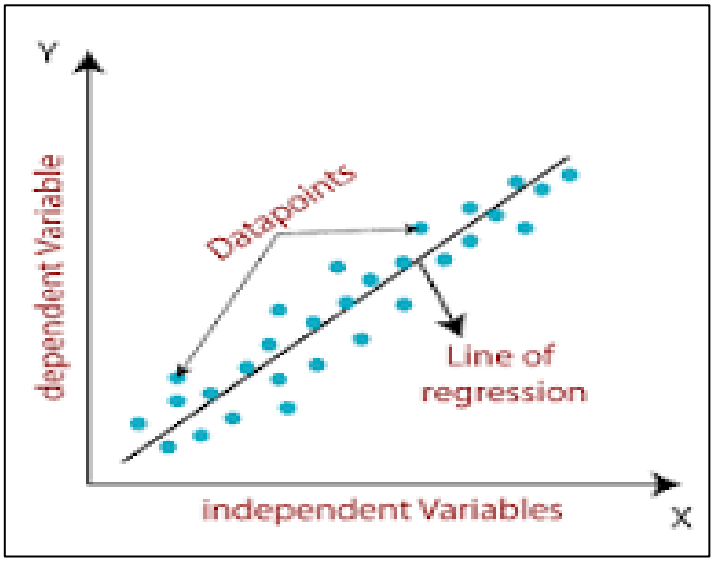
1. ***NumPy –*** The NumPy is used for scientific computing and numerical analysis and primarily used to work with large, multi – dimensional arrays and matrices through its ndarray object.
2. ***Pandas –*** The Pandas is used for data manipulation and analysis. It provides fast and flexible data structures designed to make working with labelled data.
3. ***Scikit Learn –*** A library for machine learning algorithms in python. It provides a vast collection of simple and efficient tools for data analysis, predictive modeling and statistical tasks.
4. ***XGBoost –*** A library for implementing gradient boosted decision trees. It is widely used for supervised learning tasks like regression and classification.
5. ***LightGBM –*** A high – performance, open – source gradient boosting framework developed by Microsoft. It is designed for speed, scalability, and memory efficiency, making it one of the most popular tools for handling large scale datasets in ML tasks.

Feature Scaling using Standard Scaler:

We have performed feature scaling using Standard Scaler where it normalizes the range of independent variables or features. It ensures that all features contribute equally to the model's performance, preventing features with larger magnitudes from dominating those smaller scales.

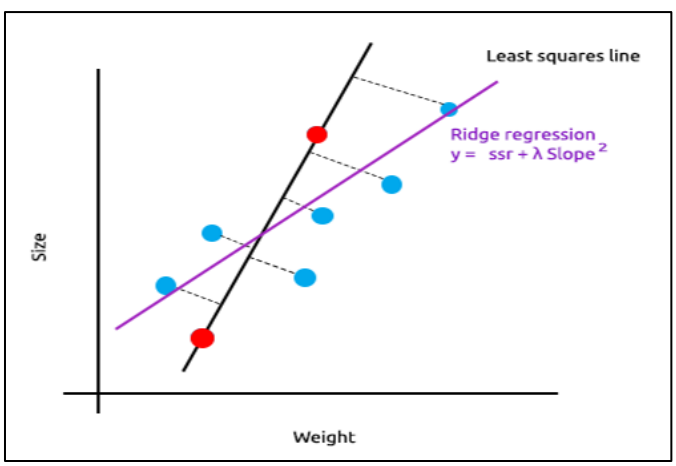
Machine Learning Algorithms which are used:

1. Linear Regression:



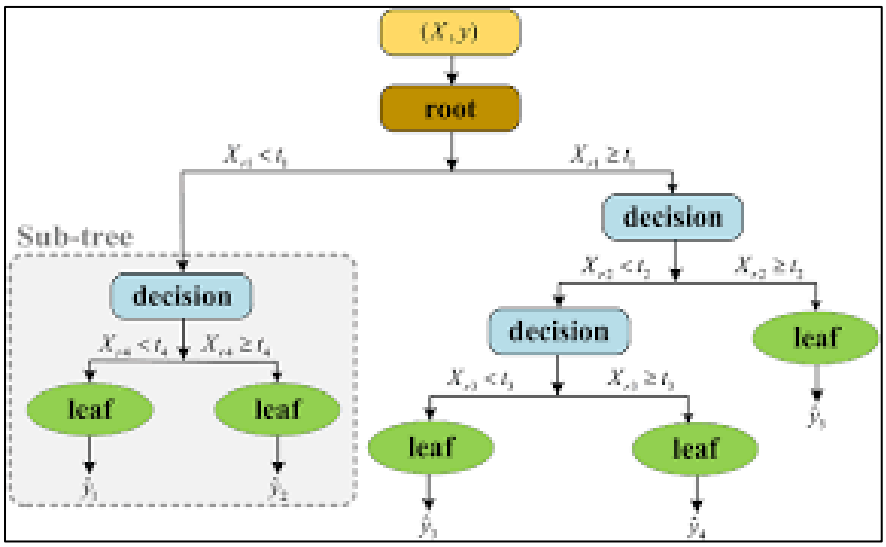
Linear Regression is a basic supervised learning algorithm used to predict continuous values like house prices. It assumes a linear relationship between input features and the target variable. The model finds the best-fit line that minimizes the error between predicted and actual values. It works using the equation of a straight line with coefficients. In implementation, the model is trained using .fit() on training data. Predictions are made using .predict() on test data. It is simple, fast, and easy to interpret. However, it may not perform well if the data is not linear.

2. Ridge Regression:



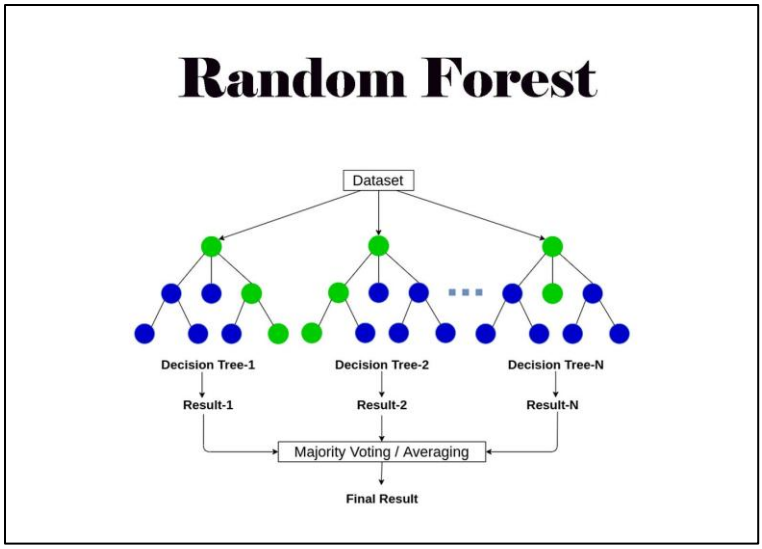
Ridge Regression is an extension of Linear Regression that helps reduce overfitting. It adds a penalty term to the loss function to shrink large coefficient values. This makes the model more stable when dealing with multicollinearity. It is especially useful when features are highly correlated. In implementation, it is similar to Linear Regression but uses a regularization parameter. The model is trained using .fit() and predictions are made using .predict(). It improves generalization on unseen data. However, it does not perform feature selection.

3. **Decision Tree Regressor:**



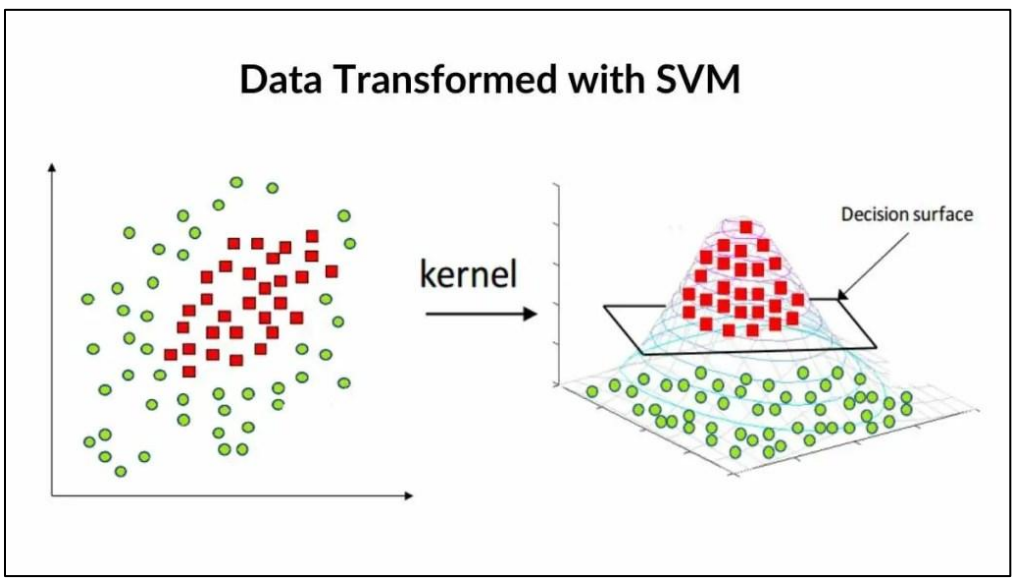
Decision Tree Regressor is a model that splits the data into smaller subsets based on feature values. It creates a tree-like structure where each node represents a decision rule. The final output is the average value of the target variable in that region. It can capture non-linear relationships in data. In implementation, the model is trained using .fit() and predictions are made using .predict(). It is easy to understand and visualize. However, it may overfit if the tree is too deep.

4. **Random Forest Regressor:**



Random Forest is an ensemble learning method that uses multiple decision trees. Each tree is trained on random subsets of data and features. The final prediction is the average of all tree outputs. This reduces overfitting and improves accuracy. It works well for complex datasets. In implementation, the model is trained using .fit() and predictions are made using .predict(). It is robust and handles large data effectively. However, it is slower compared to simple models.

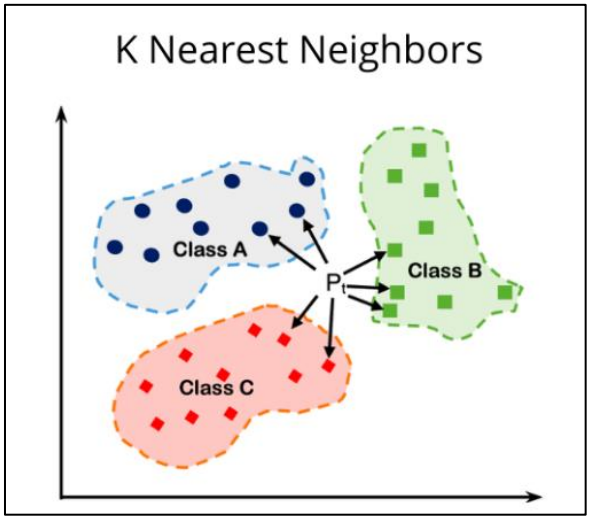
5. **Support Vector Regressor:**



SVR is a regression version of Support Vector Machine. It tries to fit the best line within a margin of tolerance (epsilon). Instead of minimizing error directly, it focuses on points within a boundary. It works well for both linear and non-linear data using kernels. In implementation, data is usually scaled before training. The model is trained using .fit()

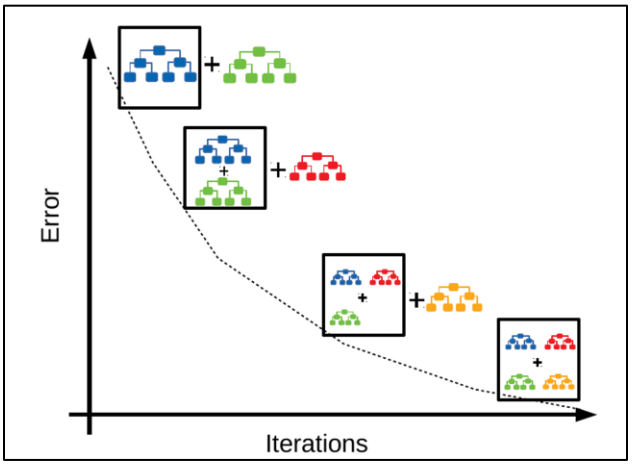
and predictions are made using .predict(). It is powerful for complex relationships. However, it is sensitive to parameter selection.

6. **KNN Regressor:**



KNN Regressor predicts values based on the average of nearest data points. It calculates the distance between data points and selects the closest neighbors. The value is computed as the mean of those neighbors. It does not build a model explicitly (lazy learning). In implementation, the model is trained using .fit() and predictions are made using .predict(). It is simple and intuitive. However, it becomes slow with large datasets and requires feature scaling.

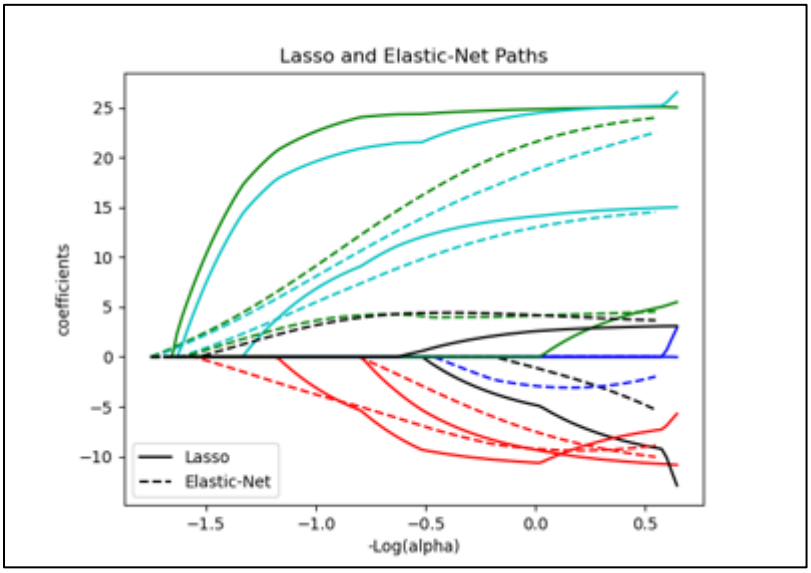
7. **Gradient Boost Regressor:**



Gradient Boosting is an advanced ensemble technique that builds models sequentially. Each new model corrects the errors of the previous one. It combines weak learners (usually decision trees) to form a strong model. It is highly effective for prediction tasks.

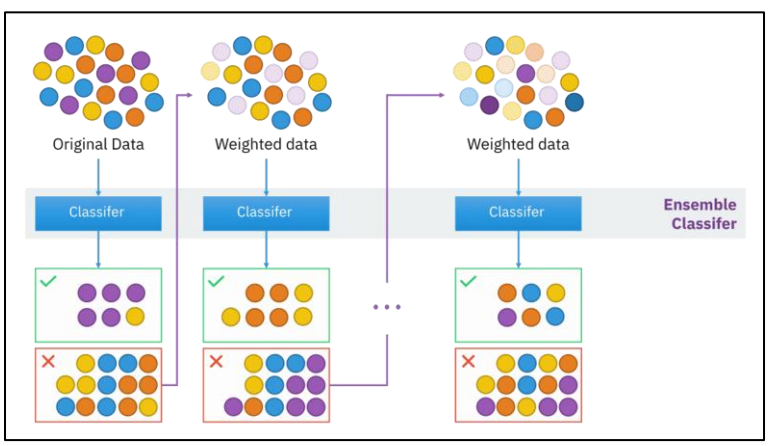
In implementation, the model is trained using .fit() and predictions are made using .predict(). It provides high accuracy and handles complex patterns well. However, it requires careful tuning and can be slow.

8. **ElasticNet Regression:**



ElasticNet is a combination of Ridge and Lasso Regression. It used both L1 and L2 regularization to improve model performance. It helps in handling multicollinearity and also performs feature selection. This makes it more balanced compared to using only Ridge or Lasso. In Implementation, we control regularization using parameters. The model is trained using .fit() and predictions are made using .predict(). It works well with many correlated features. It requires tuning of parameters.

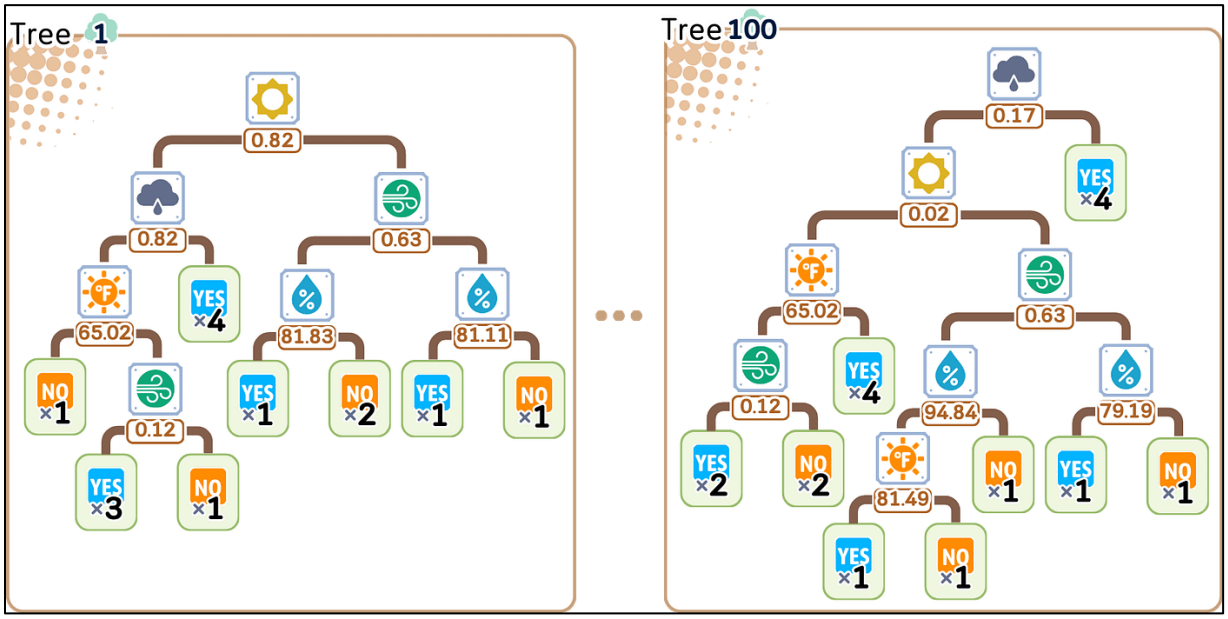
9. **AdaBoost Regressor:**



AdaBoost Regressor is an ensemble algorithm that builds models sequentially. Each new model focuses more on the errors made by previous models. It combines weak learners (usually small decision trees) to improve accuracy. It adjusts weights to give importance

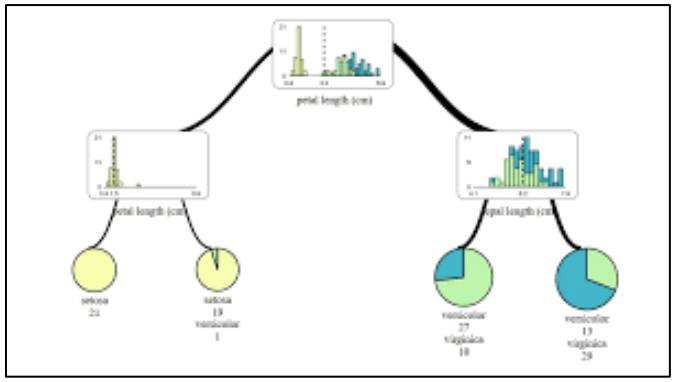
to difficult data points. In implementation, the model is trained using .fit() and predictions are made using .predict(). It improves performance of weak models. It can be sensitive to noisy data.

10. Extra Trees Regressor:



Extra Trees Regressor is also an ensemble method similar to Random Forest. It builds multiple decision trees using random splits and features. This randomness reduces variance and speeds up training. It often performs well on large datasets. In Implementation, the model is trained using .fit() and predictions are made using .predict(). It is fast and efficient. It may be less interpretable than simple models.

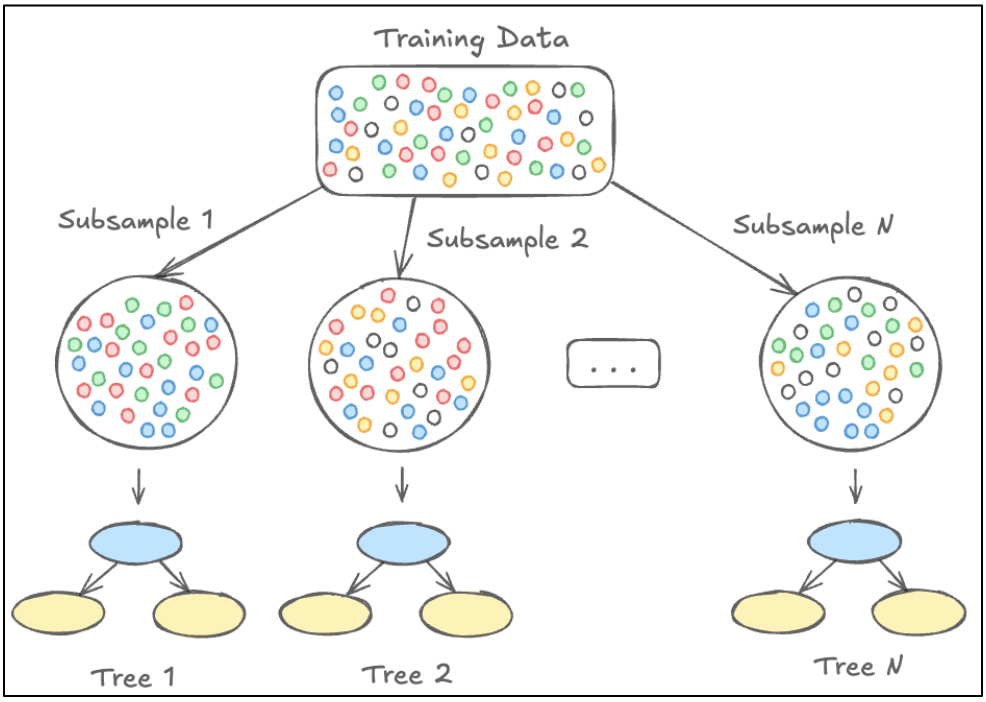
11. LightGBMRegressor:



LightGBM is an advanced gradient boosting algorithm designed for high speed and efficiency. It builds trees sequentially using a leaf – wise approach, which improves accuracy. It works well with large datasets and complex patterns. It is widely used in real – world applications. In Implementation, we use the lightGBM library. The model is trained

using .fit() and predictions are made using .predict(). It provides high performance but requires tuning.

12. XGBoost Regressor:



XGBoost (Extreme Gradient Boosting) is an advanced ensemble algorithm based on boosting. It builds models sequentially, where each new model corrects the errors of the previous ones. It uses regularization to prevent overfitting and improve performance. XGBoost is known for its speed and high accuracy. It can handle missing values and large datasets efficiently. In implementation, we used the xgboost library. It is widely used in real – world applications and competitions.

Conclusion:

In this project, we have used regression-based machine learning algorithms to predict house prices, as the target variable is continuous. Classification algorithms were not suitable since they are used for categorical outputs. We implemented different regression models such as Linear Regression, Ridge Regression, Decision Tree Regressor, Random Forest Regressor, Support Vector Regressor, KNN Regressor, Gradient Boosting, Polynomial Regression, ElasticNet Regression, AdaBoost Regressor, Extra Trees Regressor, and XGBoost Regressor, and evaluated their performance.

Each algorithm was trained on the same dataset and tested using performance metrics like MAE, MSE, and R^2 score. The results showed that ensemble methods like Random Forest, Gradient Boosting, and XGBoost generally provided better accuracy compared to simpler models. Linear models performed well for linear relationships, while tree-based models handled complex patterns effectively. The best model can be selected based on performance and application requirements.

What I Have Learnt?

I understood that we have many algorithms to solve a problem and we can use any kind of it based on the requirements. But main thing is we need to know which algorithm is best suitable for the data we have. Here we have implemented seven kinds of algorithms just to know which gives us more accuracy. Based on that we can select the best fit one. Sometimes data may become underfit or overfit based on the algorithm we select. I understood that we need to be careful while selecting the best model and it need to give us the optimum utilization of the resources with best results. The statistical measures MSE, MAE and R^2 helps us to understand the models performance by giving us a single value.